



UNIVERSIDADE DE BRASÍLIA

Faculdade de Ciências e Tecnologias em Engenharia (FCTE)

Bacharelado em Engenharia de Software

Programação para Sistemas Paralelos e Distribuídos:

Jogo da Vida

Autores

Arthur Grandão de Mello - 211039250

Bruno Martins Valério Bomfim - 211039297

Douglas Alves dos Santos - 211029620

Gabriel Campello Marques - 211039439

Geovanna Maciel Avelino da Costa - 202016328

Orientação

Prof. Dr. Fernando William Cruz

Brasília - DF

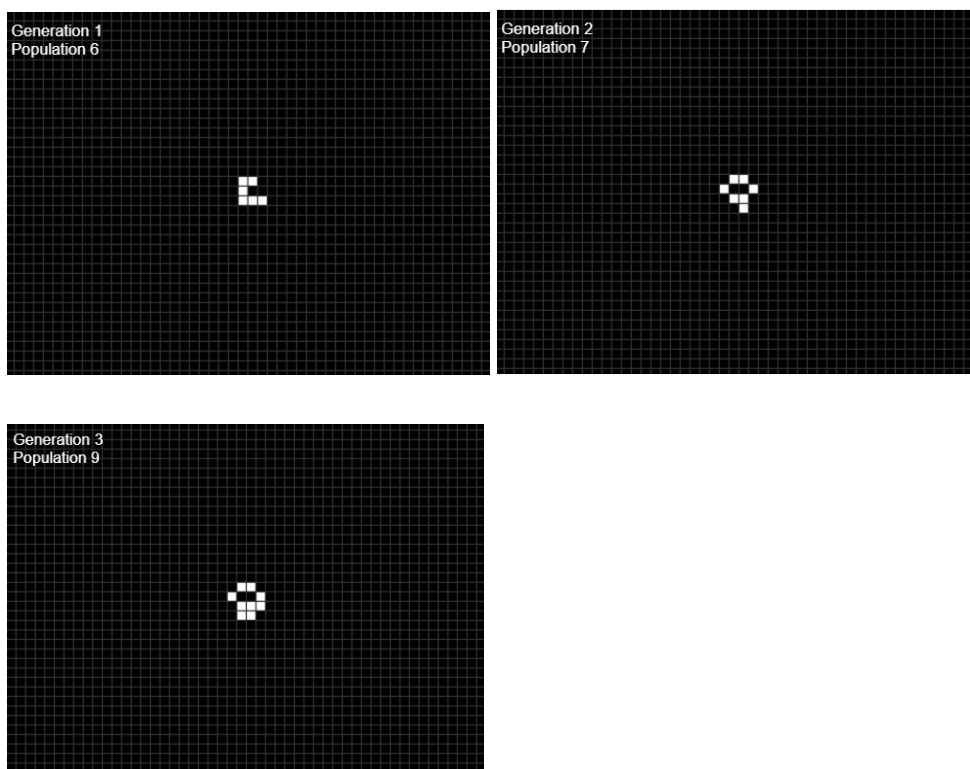
2025

Sumário

Sumário.....	2
Introdução.....	3
MPI.....	5
Implementação com MPI.....	5
Funcionamento detalhado.....	5
Divisão do tabuleiro.....	5
Inicialização distribuída.....	6
Comunicação entre processos.....	6
Atualização local (UmaVida).....	7
Sincronização.....	8
Impressão e controle.....	8
OpenMP.....	10
CUDA.....	12
Funcionamento detalhado.....	12
Paralelização com CUDA.....	12
Lógica do Kernel.....	13
Alternância de estados.....	13
Inicialização e verificação.....	14
Gestão de memória.....	14
Medição de desempenho.....	15
OpenMP com suporte a GPU.....	16
Alterações no código fornecido.....	17
1. Criação da função UmaVidaGPU com diretivas de paralelismo.....	17
2. Gerenciamento explícito da movimentação de dados com map, enter data e exit data.....	18
3. Inclusão da atualização dos dados do dispositivo para o host com update from..	18
4. Manutenção da lógica de verificação e temporização.....	19
Comparativos.....	20
Resultado do MPI.....	20
Resultado OpenMP com GPU.....	24
Resultado OpenMP.....	27
Resultado com CUDA.....	31
Conclusão do resultados.....	33
Tabela Comparativa de Tempos de Execução (Total).....	34
Referências.....	36

Introdução

O Jogo da Vida de Conway é um autômato celular desenvolvido pelo matemático britânico John Conway em 1970. Apesar de sua simplicidade, o algoritmo é capaz de gerar comportamentos complexos e caóticos a partir de regras determinísticas aplicadas a uma grade bidimensional de células. Cada célula pode estar viva ou morta, e sua evolução ao longo do tempo depende do estado das células vizinhas.



Devido à sua natureza iterativa e altamente paralelizável, o Jogo da Vida representa um excelente caso de estudo para o desenvolvimento e análise de algoritmos paralelos. Neste trabalho, exploramos três abordagens distintas de paralelização: OpenMP, MPI e CUDA, com o objetivo de acelerar a simulação em diferentes arquiteturas computacionais.

A paralelização com OpenMP permite a execução concorrente em sistemas com memória compartilhada, utilizando múltiplas threads para dividir o trabalho entre os núcleos da CPU. A abordagem com MPI foca em ambientes com memória distribuída, permitindo a comunicação entre processos independentes, o que é

ideal para clusters ou supercomputadores. Por fim, a versão com CUDA explora o poder de processamento massivo das GPUs, aproveitando milhares de núcleos para executar as atualizações celulares de forma altamente paralela.

Este trabalho apresenta a implementação, otimização e análise comparativa dessas três estratégias, discutindo seus ganhos de desempenho, limitações e adequação a diferentes contextos computacionais.

O resultado deste trabalho pode ser visualizado com mais detalhes no repositório do GitHub [gitbmvb/pspd-gameOfLife](https://github.com/gitbmvb/pspd-gameOfLife).

MPI

Implementação com MPI

A implementação com MPI (Message Passing Interface) tem como objetivo explorar o paralelismo distribuído na simulação do Jogo da Vida. Nesse modelo, o problema é dividido em múltiplos processos independentes que colaboram por meio de troca explícita de mensagens. A escolha de MPI é especialmente adequada para ambientes onde os processos não compartilham memória, como em clusters de computadores ou múltiplos nós de uma rede de alto desempenho.

No caso da simulação do Jogo da Vida, cada célula do tabuleiro depende do estado das células vizinhas. Assim, para distribuir o trabalho entre os processos, o tabuleiro bidimensional é particionado horizontalmente: cada processo fica responsável por um subconjunto contínuo de linhas. No entanto, como as células nas extremidades de cada partição dependem de vizinhos que podem estar em outro processo, é necessário realizar trocas de bordas (linhas superior e inferior) a cada iteração de atualização. Essa comunicação é realizada com chamadas `MPI_Sendrecv()`, garantindo que cada processo possua as informações necessárias antes de atualizar o estado das células.

A principal motivação para o uso de MPI nesta implementação é a capacidade de escalar a aplicação para múltiplos processos independentes, reduzindo o tempo de simulação total e distribuindo a carga de memória entre diferentes máquinas ou núcleos.

Funcionamento detalhado

Divisão do tabuleiro

O tabuleiro é dividido entre os processos de forma uniforme. Se `tam` representa a dimensão do tabuleiro e `nprocs` o número de processos, cada processo fica responsável por `loc_lines = tam / nprocs` linhas. Para evitar

problemas de arredondamento, é exigido que o número total de linhas seja divisível pelo número de processos.

Cada processo aloca seu próprio vetor local, que inclui duas linhas adicionais: uma no topo e uma na base, destinadas a armazenar as bordas recebidas dos vizinhos.

Inicialização distribuída

Cada processo inicializa sua região com células mortas. O processo de rank 0 também inicializa o padrão inicial de células vivas na sua própria região. Isso garante que o padrão inicial do Jogo da Vida esteja presente mesmo na execução distribuída.

Comunicação entre processos

Como cada célula depende de suas vizinhas, é necessário que, a cada iteração, as linhas de borda de cada processo sejam atualizadas com informações de seus vizinhos adjacentes. Isso é feito com duas operações de troca:

- O processo envia sua primeira linha de dados para o vizinho superior (rank - 1) e recebe a última linha do vizinho superior.
- Em seguida, o processo envia sua última linha de dados para o vizinho inferior (rank + 1) e recebe a primeira linha do vizinho inferior.

Essa troca é realizada com `MPI_Sendrecv()`, garantindo sincronização e atualizações corretas em todas as iterações. A imagem a seguir mostra esse processo.

```

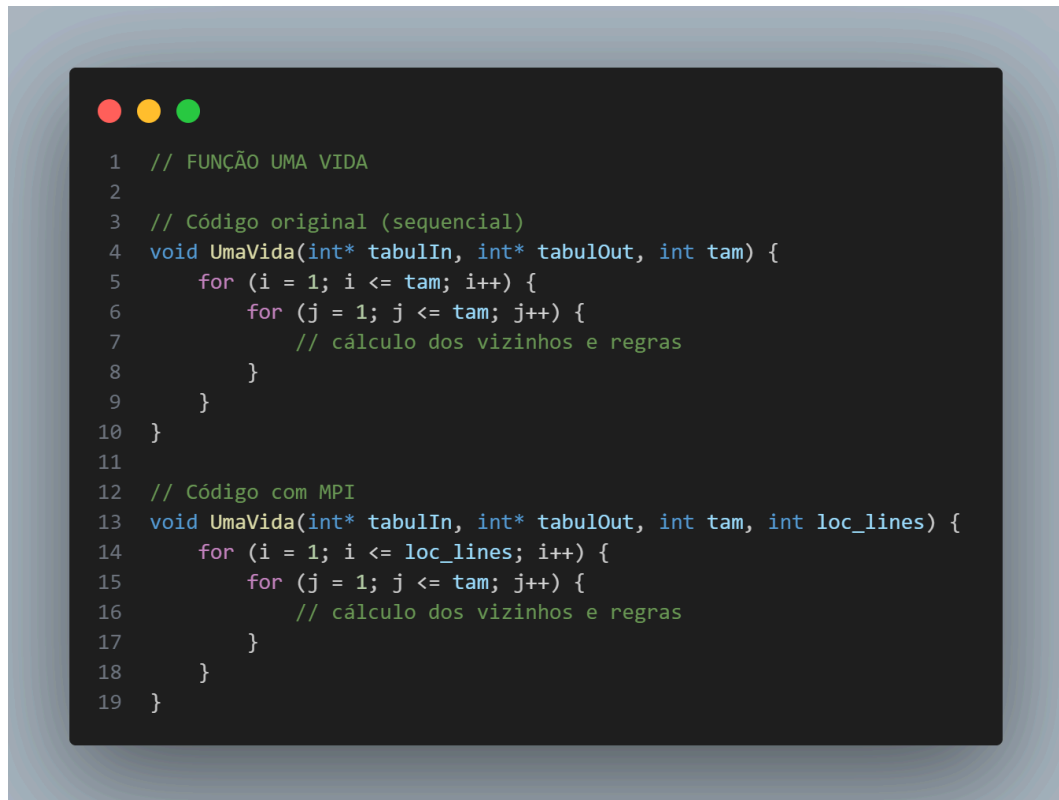
1  // COMUNICAÇÃO ENTRE OS PROCESSOS
2
3  // Código original (sequencial)
4  for (i = 0; i < 2 * (tam - 3); i++) {
5      UmaVida(tabulIn, tabulOut, tam);
6      UmaVida(tabulOut, tabulIn, tam);
7  }
8
9  // Código com MPI
10 for (int iter = 0; iter < 2 * (tam - 3); iter++) {
11     if (rank != 0)
12         MPI_Sendrecv(&tabulIn[ind2d(1,0,tam)], tam+2, MPI_INT, rank-1, 0,
13                     &tabulIn[ind2d(0,0,tam)], tam+2, MPI_INT, rank-1, 0,
14                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
15
16     if (rank != nprocs - 1)
17         MPI_Sendrecv(&tabulIn[ind2d(loc_lines+1,0,tam)], tam+2, MPI_INT, rank+1, 0,
18                     &tabulIn[ind2d(loc_lines+1,0,tam)], tam+2, MPI_INT, rank+1, 0,
19                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
20
21     UmaVida(tabulIn, tabulOut, tam, loc_lines);
22
23     // troca dos ponteiros
24     int* tmp = tabulIn;
25     tabulIn = tabulOut;
26     tabulOut = tmp;
27 }

```

Imagem 1: seção modificada do código em MPI (Fonte: Gabriel Campello).

Atualização local (UmaVida)

Após a troca de bordas, cada processo aplica a lógica do Jogo da Vida apenas sobre suas linhas locais. A atualização utiliza as informações recém-recebidas nas bordas para garantir o correto funcionamento nas extremidades da partição, como demonstrado na imagem a seguir.



```
1 // FUNÇÃO UMA VIDA
2
3 // Código original (sequencial)
4 void UmaVida(int* tabulIn, int* tabulOut, int tam) {
5     for (i = 1; i <= tam; i++) {
6         for (j = 1; j <= tam; j++) {
7             // cálculo dos vizinhos e regras
8         }
9     }
10 }
11
12 // Código com MPI
13 void UmaVida(int* tabulIn, int* tabulOut, int tam, int loc_lines) {
14     for (i = 1; i <= loc_lines; i++) {
15         for (j = 1; j <= tam; j++) {
16             // cálculo dos vizinhos e regras
17         }
18     }
19 }
```

Imagem 2: seção modificada do código em MPI (Fonte: Gabriel Campello).

Sincronização

No modelo MPI, não há sincronização automática de memória entre os processos. No entanto, a comunicação com `MPI_Sendrecv()` já garante que todos os processos possuam os dados corretos de seus vizinhos antes de prosseguir para a atualização seguinte.

Impressão e controle

A saída de resultados e o controle de tempo são centralizados no processo de rank 0, que imprime os tempos de inicialização, computação e execução total.

```

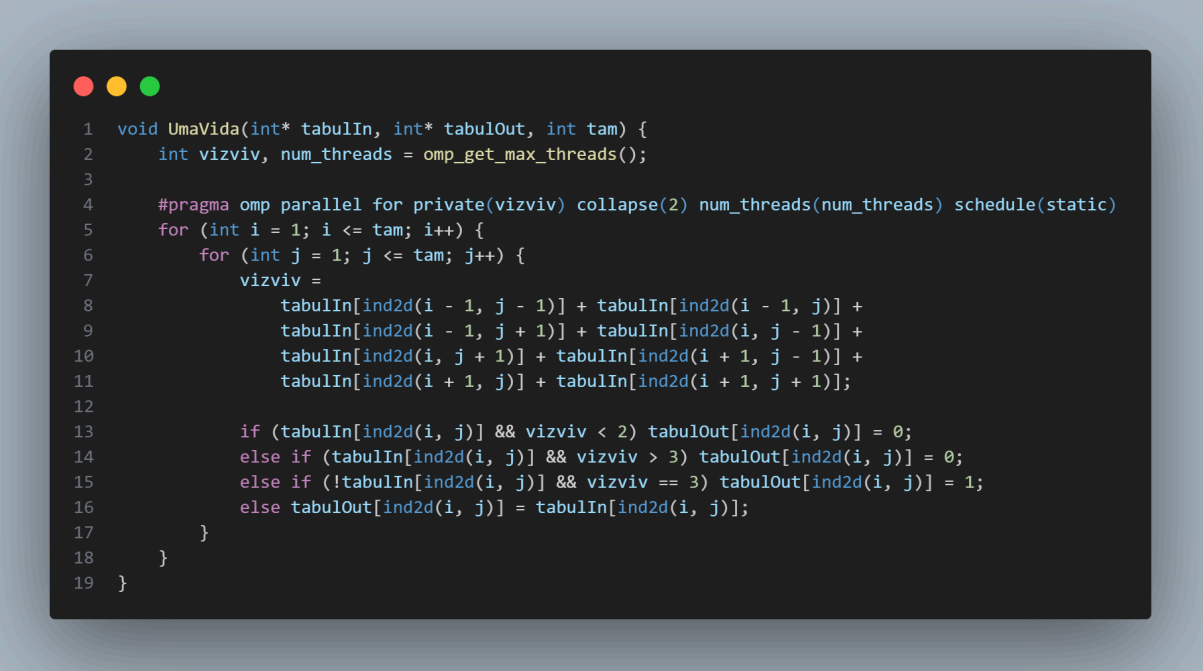
1  // INICIALIZAÇÃO
2
3  // Código original (sequencial)
4  int main(void) {
5      // ...
6      for (pow = POWMIN; pow <= POWMAX; pow++) {
7          tam = 1 << pow;
8          tabulIn = (int*) malloc((tam + 2) * (tam + 2) * sizeof(int));
9          tabulOut = (int*) malloc((tam + 2) * (tam + 2) * sizeof(int));
10         InitTabul(tabulIn, tabulOut, tam);
11         // ...
12     }
13     return 0;
14 }
15
16 // Código com MPI
17 int main(int argc, char *argv[]) {
18     MPI_Init(&argc, &argv);
19     MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
20     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
21
22     for (pow = POWMIN; pow <= POWMAX; pow++) {
23         tam = 1 << pow;
24         loc_lines = tam / nprocs;
25         if (tam % nprocs != 0) {
26             if (rank == 0) printf("Erro de divisão");
27             MPI_Finalize();
28             return 1;
29         }
30         tabulIn = (int*) malloc((loc_lines + 2) * (tam + 2) * sizeof(int));
31         tabulOut = (int*) malloc((loc_lines + 2) * (tam + 2) * sizeof(int));
32         InitTabul(tabulIn, tam, loc_lines, rank);
33         // ...
34     }
35     MPI_Finalize();
36     return 0;
37 }

```

Imagem 3: seção modificada do código em MPI (Fonte: Gabriel Campello).

OpenMP

A paralelização com OpenMP foi executada com êxito, utilizando pragmas de forma estratégica para acelerar os diversos loops presentes no código. A principal função do jogo, ***void UmaVida()***, foi modificada da seguinte maneira:



```
1 void UmaVida(int* tabulIn, int* tabulOut, int tam) {
2     int vizviv, num_threads = omp_get_max_threads();
3
4     #pragma omp parallel for private(vizviv) collapse(2) num_threads(num_threads) schedule(static)
5     for (int i = 1; i <= tam; i++) {
6         for (int j = 1; j <= tam; j++) {
7             vizviv =
8                 tabulIn[ind2d(i - 1, j - 1)] + tabulIn[ind2d(i - 1, j)] +
9                 tabulIn[ind2d(i - 1, j + 1)] + tabulIn[ind2d(i, j - 1)] +
10                tabulIn[ind2d(i, j + 1)] + tabulIn[ind2d(i + 1, j - 1)] +
11                tabulIn[ind2d(i + 1, j)] + tabulIn[ind2d(i + 1, j + 1)];
12
13            if (tabulIn[ind2d(i, j)] && vizviv < 2) tabulOut[ind2d(i, j)] = 0;
14            else if (tabulIn[ind2d(i, j)] && vizviv > 3) tabulOut[ind2d(i, j)] = 0;
15            else if (!tabulIn[ind2d(i, j)] && vizviv == 3) tabulOut[ind2d(i, j)] = 1;
16            else tabulOut[ind2d(i, j)] = tabulIn[ind2d(i, j)];
17        }
18    }
19 }
```

Imagem 4: seção modificada na versão paralela da função UmaVida (Fonte: Bruno Martins).

O parâmetro *collapse* permite interpretar um conjunto de loops aninhados como um único loop, cujo total de iterações corresponde à soma dos intervalos de atuação de cada loop individualmente. Nesse sentido, este pragma irá distribuir de forma homogênea a quantidade de iterações dos dois loops (ou seja, $\text{tam} * \text{tam}$) entre o máximo de threads especificado na variável de ambiente `OMP_NUM_THREADS`. A escolha do tipo de planejamento (*schedule*) foi *static*, tendo em vista que todas as threads serão incumbidas da mesma quantidade de trabalho.

A função de inicialização também foi paralelizada do seguinte modo:

```

1 void InitTabul(int* tabulIn, int* tabulOut, int tam) {
2     #pragma omp parallel for schedule(static)
3     for (int ij = 0; ij < (tam + 2) * (tam + 2); ij++) {
4         tabulIn[ij] = 0;
5         tabulOut[ij] = 0;
6     }
7
8     tabulIn[ind2d(1, 2)] = 1;
9     tabulIn[ind2d(2, 3)] = 1;
10    tabulIn[ind2d(3, 1)] = 1;
11    tabulIn[ind2d(3, 2)] = 1;
12    tabulIn[ind2d(3, 3)] = 1;
13 }

```

Imagem 5: seção modificada na versão paralela da função InitTabul (Fonte: Bruno Martins).

A mesma lógica foi aplicada para a função de checar os resultados. Destaque para o uso de redução na variável cnt:

```

1 int Correto(int* tabul, int tam) {
2     int cnt = 0;
3
4     #pragma omp parallel for reduction(+:cnt)
5     for (int ij = 0; ij < (tam + 2) * (tam + 2); ij++)
6         cnt += tabul[ij];
7
8     return (cnt == 5 &&
9         tabul[ind2d(tam - 2, tam - 1)] &&
10        tabul[ind2d(tam - 1, tam)] &&
11        tabul[ind2d(tam, tam - 2)] &&
12        tabul[ind2d(tam, tam - 1)] &&
13        tabul[ind2d(tam, tam)]);
14 }

```

Imagem 6: seção modificada na versão paralela da função Correto (Fonte: Bruno Martins).

CUDA

A versão **jogodavida.cu** do Jogo da Vida explora o paralelismo massivo oferecido pelas GPUs por meio da plataforma CUDA. Essa abordagem é ideal para aproveitar ao máximo a arquitetura SIMT (Single Instruction, Multiple Threads) das GPUs da NVIDIA, acelerando a simulação de forma significativa, especialmente em tabuleiros grandes.

Funcionamento detalhado

Paralelização com CUDA

O código CUDA é organizado em torno do kernel **UmaVidaKernel**, que implementa a lógica de atualização de cada célula do tabuleiro, de acordo com as regras do Jogo da Vida. Cada thread CUDA é responsável por calcular o novo estado de uma única célula do tabuleiro. Isso permite que centenas ou milhares de células sejam atualizadas simultaneamente.

A configuração do lançamento de kernel é feita com blocos 16x16 threads, valor recomendado pela própria NVIDIA, cobrindo o tabuleiro com um grid bidimensional de blocos:



```
1 dim3 block(16, 16);  
2 dim3 grid((tam + block.x - 1) / block.x, (tam + block.y - 1) / block.y);
```

Imagem 7: Detalhamento da especificação de blocos e grid (Fonte: Douglas Alves).

Lógica do Kernel

O kernel acessa diretamente os vizinhos de cada célula, acumulando o número de vizinhos vivos e aplicando as regras do jogo:

```
1  __global__ void UmaVidaKernel(int *tabIn, int *tabOut, int tam)
2  {
3      int i = blockIdx.y * blockDim.y + threadIdx.y + 1;
4      int j = blockIdx.x * blockDim.x + threadIdx.x + 1;
5
6      if (i <= tam && j <= tam)
7      {
8          int idx = ind2d(i, j, tam);
9          int vizviv =
10             tabIn[ind2d(i - 1, j - 1, tam)] + tabIn[ind2d(i - 1, j, tam)] + tabIn[ind2d(i - 1, j + 1, tam)] +
11             tabIn[ind2d(i, j - 1, tam)] + tabIn[ind2d(i, j + 1, tam)] +
12             tabIn[ind2d(i + 1, j - 1, tam)] + tabIn[ind2d(i + 1, j, tam)] + tabIn[ind2d(i + 1, j + 1, tam)];
13
14             if (tabIn[idx] && vizviv < 2)
15                 tabOut[idx] = 0;
16             else if (tabIn[idx] && vizviv > 3)
17                 tabOut[idx] = 0;
18             else if (!tabIn[idx] && vizviv == 3)
19                 tabOut[idx] = 1;
20             else
21                 tabOut[idx] = tabIn[idx];
22     }
23 }
```

Imagem 8: Demonstração da lógica do kernel (Fonte: Douglas Alves).

Alternância de estados

Cada iteração do jogo envolve duas chamadas ao kernel, alternando entre os vetores **d_in** e **d_out** para simular a troca de gerações. Isso evita cópias desnecessárias de dados e mantém a execução dentro da GPU:

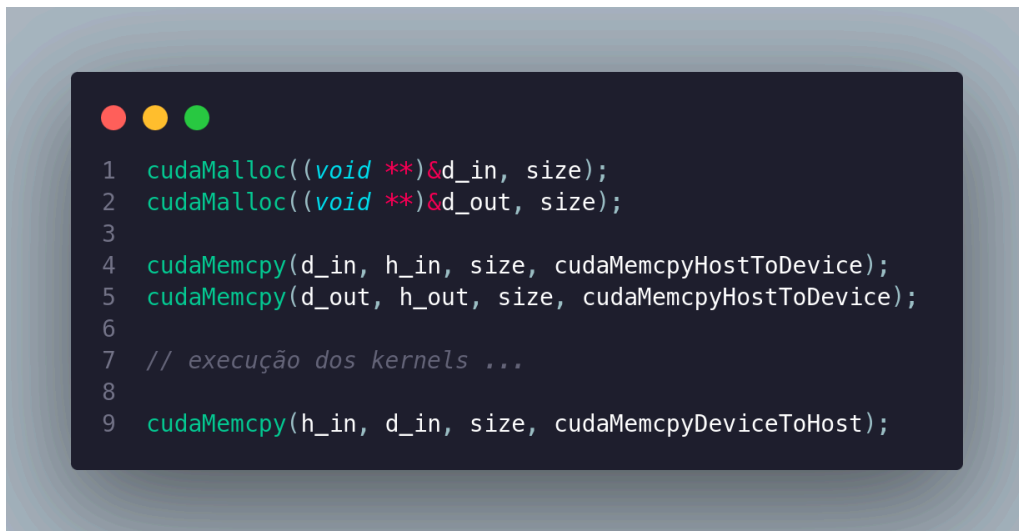
```
1  for (int i = 0; i < 2 * (tam - 3); i++)
2  {
3      UmaVidaKernel<<<grid, block>>>(d_in, d_out, tam);
4      UmaVidaKernel<<<grid, block>>>(d_out, d_in, tam);
5  }
```

Imagem 9: Chamadas do Kernel (Fonte: Douglas Alves).

Inicialização e verificação

A função **InitTabul()** configura o tabuleiro inicial com o padrão glider, que se move na diagonal. A função **Correto()** verifica se, ao final da simulação, o glider chegou ao canto inferior direito do tabuleiro — critério necessário para validar a correção da implementação.

Gestão de memória

A imagem mostra um editor de código com um fundo escuro e uma barra de título com três botões (vermelho, amarelo, verde). O código é escrito em uma linguagem de programação, provavelmente C++, e trata-se de exemplos de uso de CUDA para gestão de memória. O código está organizado em linhas numeradas de 1 a 9. As linhas 1 e 2 mostram a alocação de memória na GPU usando cudaMalloc. A linha 3 está vazia. As linhas 4 e 5 mostram a cópia de dados da CPU para a GPU usando cudaMemcpyHostToDevice. A linha 6 está vazia. A linha 7 é um comentário em português: // execução dos kernels A linha 8 está vazia. A linha 9 mostra a cópia de dados da GPU de volta para a CPU usando cudaMemcpyDeviceToHost.

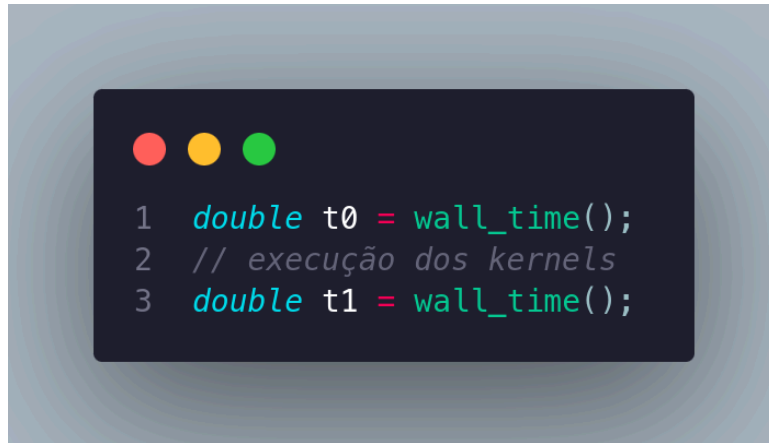
```
1  cudaMalloc((void **)&d_in, size);
2  cudaMalloc((void **)&d_out, size);
3
4  cudaMemcpy(d_in, h_in, size, cudaMemcpyHostToDevice);
5  cudaMemcpy(d_out, h_out, size, cudaMemcpyHostToDevice);
6
7  // execução dos kernels ...
8
9  cudaMemcpy(h_in, d_in, size, cudaMemcpyDeviceToHost);
```

Imagem 10: Gestão da memória com CUDA (Fonte: Douglas Alves).

- Alocação de memória na GPU com cudaMalloc;
- Cópia dos dados entre CPU e GPU com cudaMemcpy antes e após o processamento;
- Liberação de memória com cudaFree.

Medição de desempenho

O tempo de execução é medido com a função **wall_time()** antes e após o loop de iteração. Isso permite comparar o desempenho com outras versões (MPI, OpenMP, OpenMP Target) sob as mesmas condições.

A code snippet is displayed within a dark-themed window with three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in a syntax-highlighted style with line numbers 1, 2, and 3 on the left. Line 1: `double t0 = wall_time();` (double in blue, t0 in white, = in white, wall_time in green, ; in white). Line 2: `// execução dos kernels` (comment in grey). Line 3: `double t1 = wall_time();` (double in blue, t1 in white, = in white, wall_time in green, ; in white).

```
1  double t0 = wall_time();
2  // execução dos kernels
3  double t1 = wall_time();
```

Imagem 11: Medição do tempo (Fonte: Douglas Alves).

OpenMP com suporte a GPU

O OpenMP introduziu, a partir da versão 4.0, a possibilidade de offloading para dispositivos aceleradores como GPUs, usando a diretiva **#pragma omp target**. Com isso, é possível aproveitar o poder de processamento paralelo massivo das GPUs para acelerar cálculos intensivos, como no Jogo da Vida.

As principais diretivas OpenMP para GPU são:

- **#pragma omp target**: marca o bloco de código que será enviado para a GPU.
- **#pragma omp target data**: garante que dados estejam no dispositivo durante um certo escopo.
- **#pragma omp target teams**: cria grupos de threads (times), mais leves que as threads tradicionais.
- **#pragma omp distribute**: distribui trabalho entre as equipes.
- **#pragma omp parallel for**: executa um loop em paralelo. Pode ser aninhado com target teams.

Exemplo básico:



```
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main() {
5      int valor = 42;
6
7      // Envia o bloco para a GPU
8      #pragma omp target
9      {
10         printf("Olá, mundo da GPU! valor = %d\n", valor);
11     }
12
13     return 0;
14 }
```

O código original fornecido implementa o “Jogo da Vida” proposto por John Conway de forma sequencial, utilizando a linguagem C. O programa simula a

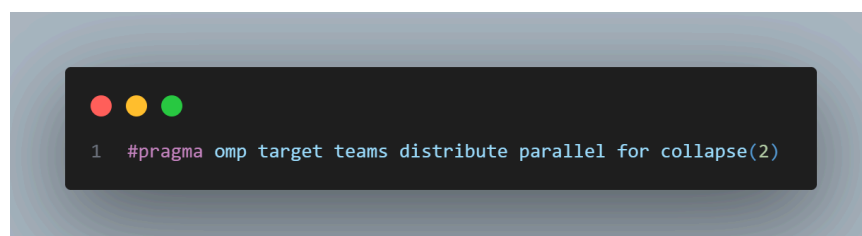
evolução de um conjunto de células vivas e mortas em um tabuleiro bidimensional de tamanho variável, aplicando regras simples de vizinhança a cada geração. A estrutura do código original é bastante didática: utiliza dois vetores para representar os estados do tabuleiro e alterna entre eles ao longo das gerações, aplicando as regras definidas para nascimento, morte e sobrevivência celular. Essa versão, no entanto, não explora nenhuma forma de paralelismo e executa toda a computação na CPU, o que limita significativamente seu desempenho, especialmente para tabuleiros de grande dimensão.

O principal objetivo das modificações realizadas neste trabalho foi **adaptar o código sequencial original para que a lógica de evolução das gerações fosse executada em uma GPU**, utilizando **OpenMP com diretivas Target**. Essa estratégia visa explorar o poder de paralelismo massivo oferecido pelas arquiteturas de GPU, sem recorrer a linguagens ou APIs complexas como CUDA. A escolha por OpenMP se justifica pela simplicidade da sua sintaxe, ampla adoção em ambientes acadêmicos e sua portabilidade entre diferentes arquiteturas.

Alterações no código fornecido

1. Criação da função UmaVidaGPU com diretivas de paralelismo

A função UmaVida, responsável por calcular uma geração do tabuleiro, foi substituída por uma nova versão chamada UmaVidaGPU. Nessa nova função, os laços que percorrem cada célula do tabuleiro foram paralelizados utilizando a diretiva:



```
1 #pragma omp target teams distribute parallel for collapse(2)
```

Essa diretiva informa ao compilador que o bloco de código a seguir deve ser executado na GPU, distribuindo as iterações dos laços for entre diferentes equipes e threads da arquitetura paralela. O modificador collapse(2) foi utilizado para que

os dois laços aninhados (que percorrem as dimensões do tabuleiro) sejam combinados em um único espaço de iteração paralela, permitindo melhor aproveitamento da hierarquia de paralelismo da GPU.

2. Gerenciamento explícito da movimentação de dados com `map`, `enter data` e `exit data`

Ao se trabalhar com GPUs, é essencial controlar a movimentação de dados entre a memória do host (CPU) e a memória do dispositivo (GPU). Para isso, foram utilizadas as diretivas:

```
map(to: ...) map(from: ...)
```

Essas diretivas foram inseridas tanto na função `UmaVidaGPU` quanto em blocos separados de preparação e limpeza de dados:

```
#pragma omp target enter data map(to: tabulIn[...],  
tabulOut[...])
```

```
#pragma omp target exit data map(delete: tabulIn[...],  
tabulOut[...])
```

Essas chamadas garantem que os vetores do tabuleiro sejam **enviados uma única vez à GPU** antes do início da simulação e **removidos ao final**, evitando que sejam copiados repetidamente a cada iteração, o que comprometeria o desempenho. Essa estratégia de gestão de dados manual é uma **boa prática de programação com OpenMP Target**, pois evita overheads desnecessários.

3. Inclusão da atualização dos dados do dispositivo para o host com `update from`

Após a execução de todas as gerações na GPU, é necessário que os dados resultantes sejam copiados de volta para a CPU para que a verificação de corretude possa ser realizada. Para isso, foi utilizada a diretiva:

```
#pragma omp target update from(tabulIn[...])
```

Essa linha garante que o vetor `tabulln`, que contém o estado final do tabuleiro, seja sincronizado com a memória da CPU para que a função `Correto()` possa verificar se o glider chegou corretamente ao canto inferior direito do tabuleiro.

4. Manutenção da lógica de verificação e temporização

A função `Correto` foi mantida exatamente como na versão original, pois ela verifica se o glider, após um número determinado de gerações, se posicionou corretamente no canto inferior direito do tabuleiro. Essa função é essencial para garantir que a simulação está correta, mesmo após a paralelização e uso de hardware acelerado. Da mesma forma, o uso da função `wall_time()` para medir o tempo de execução foi mantido, permitindo comparar o desempenho entre diferentes tamanhos de tabuleiro.

Abaixo estão os resultados da versão do jogo da vida com OpenMP Target (GPU):

```
manuzini@ManuGames:~/faculdade/pspd$ ./jogodavida
**RESULTADO CORRETO**
tam=8; tempos: init=0.0000060, comp=0.0000110, fim=0.0000231, tot=0.0000401
**RESULTADO CORRETO**
tam=16; tempos: init=0.0000010, comp=0.0000911, fim=0.0000110, tot=0.0001030
**RESULTADO CORRETO**
tam=32; tempos: init=0.0000021, comp=0.0007880, fim=0.0000129, tot=0.0008030
**RESULTADO CORRETO**
tam=64; tempos: init=0.0000119, comp=0.0058830, fim=0.0000319, tot=0.0059268
**RESULTADO CORRETO**
tam=128; tempos: init=0.0000701, comp=0.0536458, fim=0.0000570, tot=0.0537729
**RESULTADO CORRETO**
tam=256; tempos: init=0.0002880, comp=0.4009969, fim=0.0001621, tot=0.4014471
**RESULTADO CORRETO**
tam=512; tempos: init=0.0010440, comp=3.2180710, fim=0.0006959, tot=3.2198110
**RESULTADO CORRETO**
tam=1024; tempos: init=0.0056801, comp=31.0015979, fim=0.0019901, tot=31.0092680
```

Em resumo, as modificações realizadas transformam um código sequencial em uma aplicação paralela com aceleração via GPU, utilizando as diretivas do OpenMP Target. Essa abordagem traz ganhos significativos de desempenho com baixo custo de complexidade, sendo ideal tanto para aplicações acadêmicas

quanto para protótipos científicos. O trabalho reforça a viabilidade do uso de OpenMP como alternativa simples e poderosa ao CUDA e ao MPI em contextos de computação de alto desempenho com foco em um único nó com GPU.

Comparativos

Resultado do MPI

Resultados da execução MPI - sex 20 jun 2025 21:01:33 -03

=====

Executando com 1 processo(s)

>>> Saída para mpirun -np 1

```
tam=8; tempos: init=0.0000021, comp=0.0000091, total=0.0000112
tam=16; tempos: init=0.0000012, comp=0.0000789, total=0.0000801
tam=32; tempos: init=0.0000041, comp=0.0007501, total=0.0007541
tam=64; tempos: init=0.0000150, comp=0.0056510, total=0.0056660
tam=128; tempos: init=0.0000730, comp=0.0529661, total=0.0530391
tam=256; tempos: init=0.0003171, comp=0.4245250, total=0.4248421
tam=512; tempos: init=0.0012429, comp=3.1795549, total=3.1807978
tam=1024; tempos: init=0.0049329, comp=27.1834810, total=27.1884139
```

Executando com 2 processo(s)

>>> Saída para mpirun -np 2

```
tam=8; tempos: init=0.0000031, comp=0.0000169, total=0.0000200
tam=16; tempos: init=0.0000000, comp=0.0001590, total=0.0001590
tam=32; tempos: init=0.0000021, comp=0.0013959, total=0.0013981
tam=64; tempos: init=0.0000110, comp=0.0095840, total=0.0095949
tam=128; tempos: init=0.0000739, comp=0.0495379, total=0.0496118
tam=256; tempos: init=0.0003519, comp=0.3765991, total=0.3769510
tam=512; tempos: init=0.0012379, comp=3.4576850, total=3.4589229
tam=1024; tempos: init=0.0046730, comp=27.2429321, total=27.2476051
tam=8; tempos: init=0.0000038, comp=0.0000122, total=0.0000160
tam=16; tempos: init=0.0000010, comp=0.0001130, total=0.0001140
tam=32; tempos: init=0.0000031, comp=0.0009849, total=0.0009880
tam=64; tempos: init=0.0000110, comp=0.0107272, total=0.0107381
```

```
tam=128; tempos: init=0.0000648, comp=0.0528591, total=0.0529239
tam=256; tempos: init=0.0003481, comp=0.4438419, total=0.4441900
tam=512; tempos: init=0.0012960, comp=3.4481251, total=3.4494212
tam=1024; tempos: init=0.0044789, comp=27.4563720, total=27.4608510
```

Executando com 4 processo(s)

>>> Saída para mpirun -np 4

```
tam=8; tempos: init=0.0000029, comp=0.0000100, total=0.0000129
tam=16; tempos: init=0.0000010, comp=0.0000870, total=0.0000880
tam=32; tempos: init=0.0000041, comp=0.0008030, total=0.0008070
tam=64; tempos: init=0.0000188, comp=0.0062320, total=0.0062509
tam=128; tempos: init=0.0000830, comp=0.0518610, total=0.0519440
tam=256; tempos: init=0.0003741, comp=0.4792540, total=0.4796281
tam=512; tempos: init=0.0011730, comp=3.7598860, total=3.7610590
tam=1024; tempos: init=0.0049510, comp=29.3419099, total=29.3468609
tam=8; tempos: init=0.0000031, comp=0.0000100, total=0.0000131
tam=16; tempos: init=0.0000012, comp=0.0000880, total=0.0000892
tam=32; tempos: init=0.0000029, comp=0.0007472, total=0.0007501
tam=64; tempos: init=0.0000160, comp=0.0062630, total=0.0062790
tam=128; tempos: init=0.0000851, comp=0.0518999, total=0.0519850
tam=256; tempos: init=0.0003781, comp=0.4108460, total=0.4112241
tam=512; tempos: init=0.0013809, comp=3.7739360, total=3.7753170
tam=1024; tempos: init=0.0053949, comp=29.7311089, total=29.7365038
tam=8; tempos: init=0.0000019, comp=0.0000122, total=0.0000141
tam=16; tempos: init=0.0000000, comp=0.0001068, total=0.0001068
tam=32; tempos: init=0.0000019, comp=0.0009220, total=0.0009239
tam=64; tempos: init=0.0000122, comp=0.0078030, total=0.0078151
tam=128; tempos: init=0.0000639, comp=0.0527451, total=0.0528090
tam=256; tempos: init=0.0003660, comp=0.4105780, total=0.4109440
tam=512; tempos: init=0.0013630, comp=3.7652469, total=3.7666099
tam=1024; tempos: init=0.0047779, comp=29.8821909, total=29.8869689
tam=8; tempos: init=0.0000021, comp=0.0000110, total=0.0000131
tam=16; tempos: init=0.0000010, comp=0.0000889, total=0.0000899
tam=32; tempos: init=0.0000031, comp=0.0007880, total=0.0007911
tam=64; tempos: init=0.0000160, comp=0.0062540, total=0.0062699
tam=128; tempos: init=0.0000799, comp=0.0520432, total=0.0521231
```

```
tam=256; tempos: init=0.0003748, comp=0.4104140, total=0.4107888
tam=512; tempos: init=0.0013690, comp=3.7590580, total=3.7604270
tam=1024; tempos: init=0.0051270, comp=30.0795701, total=30.0846970
```

Executando com 8 processo(s)

>>> Saída para mpirun -np 8

```
tam=8; tempos: init=0.0000031, comp=0.0000129, total=0.0000160
tam=16; tempos: init=0.0000010, comp=0.0001271, total=0.0001280
tam=32; tempos: init=0.0000019, comp=0.0010960, total=0.0010979
tam=64; tempos: init=0.0000172, comp=0.0118060, total=0.0118232
tam=128; tempos: init=0.0000701, comp=0.1039639, total=0.1040339
tam=256; tempos: init=0.0003679, comp=0.4381261, total=0.4384940
tam=512; tempos: init=0.0014699, comp=3.5291350, total=3.5306048
tam=1024; tempos: init=0.0059390, comp=32.3024950, total=32.3084340
tam=8; tempos: init=0.0000031, comp=0.0000191, total=0.0000222
tam=16; tempos: init=0.0000010, comp=0.0001781, total=0.0001791
tam=32; tempos: init=0.0000031, comp=0.0015690, total=0.0015721
tam=64; tempos: init=0.0000141, comp=0.0132320, total=0.0132461
tam=128; tempos: init=0.0000701, comp=0.1003499, total=0.1004200
tam=256; tempos: init=0.0003660, comp=0.5141098, total=0.5144758
tam=512; tempos: init=0.0014579, comp=4.0672281, total=4.0686860
tam=1024; tempos: init=0.0055180, comp=32.0160470, total=32.0215650
tam=8; tempos: init=0.0000041, comp=0.0000141, total=0.0000181
tam=16; tempos: init=0.0000012, comp=0.0001259, total=0.0001271
tam=32; tempos: init=0.0000029, comp=0.0011191, total=0.0011220
tam=64; tempos: init=0.0000169, comp=0.0070121, total=0.0070291
tam=128; tempos: init=0.0000832, comp=0.0532508, total=0.0533340
tam=256; tempos: init=0.0003700, comp=0.4391069, total=0.4394770
tam=512; tempos: init=0.0014369, comp=4.0909660, total=4.0924029
tam=1024; tempos: init=0.0054419, comp=32.2553110, total=32.2607529
tam=8; tempos: init=0.0000019, comp=0.0000112, total=0.0000131
tam=16; tempos: init=0.0000010, comp=0.0000930, total=0.0000939
tam=32; tempos: init=0.0000038, comp=0.0008011, total=0.0008049
tam=64; tempos: init=0.0000210, comp=0.0066881, total=0.0067091
tam=128; tempos: init=0.0000842, comp=0.0531440, total=0.0532281
tam=256; tempos: init=0.0003650, comp=0.4345999, total=0.4349649
```

tam=512; tempos: init=0.0013890, comp=4.1233530, total=4.1247420
tam=1024; tempos: init=0.0053959, comp=32.4618809, total=32.4672768
tam=8; tempos: init=0.0000019, comp=0.0000110, total=0.0000129
tam=16; tempos: init=0.0000010, comp=0.0000942, total=0.0000951
tam=32; tempos: init=0.0000041, comp=0.0007999, total=0.0008039
tam=64; tempos: init=0.0000169, comp=0.0066330, total=0.0066500
tam=128; tempos: init=0.0000830, comp=0.0617712, total=0.0618541
tam=256; tempos: init=0.0003660, comp=0.4952819, total=0.4956479
tam=512; tempos: init=0.0014660, comp=4.0061650, total=4.0076311
tam=1024; tempos: init=0.0051498, comp=32.7636080, total=32.7687578
tam=8; tempos: init=0.0000031, comp=0.0000098, total=0.0000129
tam=16; tempos: init=0.0000010, comp=0.0000939, total=0.0000949
tam=32; tempos: init=0.0000041, comp=0.0008001, total=0.0008042
tam=64; tempos: init=0.0000169, comp=0.0067031, total=0.0067201
tam=128; tempos: init=0.0000849, comp=0.0530150, total=0.0530999
tam=256; tempos: init=0.0003650, comp=0.5111089, total=0.5114739
tam=512; tempos: init=0.0014520, comp=4.0725789, total=4.0740309
tam=1024; tempos: init=0.0052938, comp=32.8352711, total=32.8405650
tam=8; tempos: init=0.0000031, comp=0.0000100, total=0.0000131
tam=16; tempos: init=0.0000010, comp=0.0000911, total=0.0000920
tam=32; tempos: init=0.0000038, comp=0.0007961, total=0.0007999
tam=64; tempos: init=0.0000169, comp=0.0066841, total=0.0067010
tam=128; tempos: init=0.0000920, comp=0.0536759, total=0.0537679
tam=256; tempos: init=0.0003700, comp=0.5144749, total=0.5148449
tam=512; tempos: init=0.0012879, comp=4.1250801, total=4.1263680
tam=1024; tempos: init=0.0053210, comp=33.0632269, total=33.0685480
tam=8; tempos: init=0.0000031, comp=0.0000119, total=0.0000150
tam=16; tempos: init=0.0000012, comp=0.0001130, total=0.0001142
tam=32; tempos: init=0.0000029, comp=0.0009792, total=0.0009820
tam=64; tempos: init=0.0000160, comp=0.0081580, total=0.0081739
tam=128; tempos: init=0.0000670, comp=0.0536270, total=0.0536940
tam=256; tempos: init=0.0003679, comp=0.4345810, total=0.4349489
tam=512; tempos: init=0.0014780, comp=4.0803280, total=4.0818059
tam=1024; tempos: init=0.0054691, comp=33.2415221, total=33.2469912

Resultado OpenMP com GPU

Resultados da execução - sex 20 jun 2025 21:04:46 -03

=====

Executando com OMP_NUM_THREADS=1

>>> Saída para OMP_NUM_THREADS=1

RESULTADO CORRETO

tam=8;	tempos:	init=0.0001411,	comp=0.0000300,	fim=0.0012209,
tot=0.0013921				

RESULTADO CORRETO

tam=16;	tempos:	init=0.0000031,	comp=0.0001700,	fim=0.0000000,
tot=0.0001731				

RESULTADO CORRETO

tam=32;	tempos:	init=0.0000050,	comp=0.0013041,	fim=0.0000029,
tot=0.0013120				

RESULTADO CORRETO

tam=64;	tempos:	init=0.0000229,	comp=0.0104201,	fim=0.0000100,
tot=0.0104530				

RESULTADO CORRETO

tam=128;	tempos:	init=0.0000989,	comp=0.0843160,	fim=0.0000360,
tot=0.0844510				

RESULTADO CORRETO

tam=256;	tempos:	init=0.0004451,	comp=0.6812429,	fim=0.0001431,
tot=0.6818311				

RESULTADO CORRETO

tam=512;	tempos:	init=0.0019100,	comp=5.4240589,	fim=0.0005710,
tot=5.4265399				

RESULTADO CORRETO

tam=1024;	tempos:	init=0.0079949,	comp=43.4556332,	fim=0.0022669,
tot=43.4658949				

Executando com OMP_NUM_THREADS=2

>>> Saída para OMP_NUM_THREADS=2

RESULTADO CORRETO

tam=8;	tempos:	init=0.0001459,	comp=0.0014560,	fim=0.0019550,
tot=0.0035570				

RESULTADO CORRETO

tam=16;	tempos:	init=0.0000019,	comp=0.0018342,	fim=0.0000019,
tot=0.0018380				

RESULTADO CORRETO

tam=32; tempos: init=0.0000119, comp=0.0046451, fim=0.0000038,
tot=0.0046608

RESULTADO CORRETO

tam=64; tempos: init=0.0000360, comp=0.0204971, fim=0.0000088,
tot=0.0205419

RESULTADO CORRETO

tam=128; tempos: init=0.0001199, comp=0.1389990, fim=0.0000372,
tot=0.1391561

RESULTADO CORRETO

tam=256; tempos: init=0.0004599, comp=0.7309680, fim=0.0001531,
tot=0.7315810

RESULTADO CORRETO

tam=512; tempos: init=0.0020921, comp=4.2513359, fim=0.0008202,
tot=4.2542481

RESULTADO CORRETO

tam=1024; tempos: init=0.0121369, comp=38.8053281, fim=0.0023689,
tot=38.8198340

Executando com OMP_NUM_THREADS=4

>>> Saída para OMP_NUM_THREADS=4

RESULTADO CORRETO

tam=8; tempos: init=0.0002091, comp=0.0023389, fim=0.0020421,
tot=0.0045900

RESULTADO CORRETO

tam=16; tempos: init=0.0000031, comp=0.0044260, fim=0.0000010,
tot=0.0044301

RESULTADO CORRETO

tam=32; tempos: init=0.0000150, comp=0.0112960, fim=0.0000041,
tot=0.0113151

RESULTADO CORRETO

tam=64; tempos: init=0.0000460, comp=0.0253601, fim=0.0000119,
tot=0.0254180

RESULTADO CORRETO

tam=128; tempos: init=0.0001640, comp=0.0922861, fim=0.0000420,
tot=0.0924921

RESULTADO CORRETO

tam=256; tempos: init=0.0005229, comp=0.4934821, fim=0.0002410,
tot=0.4942460

RESULTADO CORRETO

tam=512; tempos: init=0.0026441, comp=3.6863539, fim=0.0006011,
tot=3.6895990

RESULTADO CORRETO

tam=1024; tempos: init=0.0088410, comp=33.8216169, fim=0.0032461,
tot=33.8337040

Executando com OMP_NUM_THREADS=8

>>> Saída para OMP_NUM_THREADS=8

RESULTADO CORRETO

tam=8; tempos: init=0.0004611, comp=0.0079360, fim=0.0023539,
tot=0.0107510

RESULTADO CORRETO

tam=16; tempos: init=0.0000041, comp=0.0252900, fim=0.0000029,
tot=0.0252969

RESULTADO CORRETO

tam=32; tempos: init=0.0000191, comp=0.0374420, fim=0.0000031,
tot=0.0374641

RESULTADO CORRETO

tam=64; tempos: init=0.0000470, comp=0.0825789, fim=0.0000172,
tot=0.0826430

RESULTADO CORRETO

tam=128; tempos: init=0.0001609, comp=0.1871650, fim=0.0000491,
tot=0.1873751

RESULTADO CORRETO

tam=256; tempos: init=0.0006011, comp=0.5759621, fim=0.0001678,
tot=0.5767310

RESULTADO CORRETO

tam=512; tempos: init=0.0026591, comp=2.8409359, fim=0.0006142,
tot=2.8442092

RESULTADO CORRETO

tam=1024; tempos: init=0.0078111, comp=18.5580089, fim=0.0023952,
tot=18.5682151

Executando com OMP_NUM_THREADS=16

>>> Saída para OMP_NUM_THREADS=16

RESULTADO CORRETO

tam=8; tempos: init=0.0001500, comp=0.0349810, fim=0.0019939,
tot=0.0371249

RESULTADO CORRETO

```

tam=16;   tempos:   init=0.0000041,   comp=0.0676610,   fim=0.0000019,
tot=0.0676670
**RESULTADO CORRETO**

tam=32;   tempos:   init=0.0000141,   comp=0.1330929,   fim=0.0000050,
tot=0.1331120
**RESULTADO CORRETO**

tam=64;   tempos:   init=0.0000479,   comp=0.2888141,   fim=0.0000169,
tot=0.2888789
**RESULTADO CORRETO**

tam=128;   tempos:   init=0.0001769,   comp=0.6163340,   fim=0.0000510,
tot=0.6165619
**RESULTADO CORRETO**

tam=256;   tempos:   init=0.0006812,   comp=1.2662799,   fim=0.0001969,
tot=1.2671580
**RESULTADO CORRETO**

tam=512;   tempos:   init=0.0027480,   comp=2.9732900,   fim=0.0007141,
tot=2.9767520
**RESULTADO CORRETO**

tam=1024;   tempos:   init=0.0114150,   comp=11.4026349,   fim=0.0027680,
tot=11.4168179

```

Resultado OpenMP

Resultados da execução - sex 20 jun 2025 21:04:46 -03

=====

Executando com OMP_NUM_THREADS=1

>>> Saída para OMP_NUM_THREADS=1

RESULTADO CORRETO

```

tam=8;   tempos:   init=0.0001411,   comp=0.0000300,   fim=0.0012209,
tot=0.0013921

```

RESULTADO CORRETO

```

tam=16;   tempos:   init=0.0000031,   comp=0.0001700,   fim=0.0000000,
tot=0.0001731

```

RESULTADO CORRETO

```

tam=32;   tempos:   init=0.0000050,   comp=0.0013041,   fim=0.0000029,
tot=0.0013120

```

RESULTADO CORRETO

```

tam=64;   tempos:   init=0.0000229,   comp=0.0104201,   fim=0.0000100,
tot=0.0104530

```

RESULTADO CORRETO

tam=128; tempos: init=0.0000989, comp=0.0843160, fim=0.0000360,
tot=0.0844510

RESULTADO CORRETO

tam=256; tempos: init=0.0004451, comp=0.6812429, fim=0.0001431,
tot=0.6818311

RESULTADO CORRETO

tam=512; tempos: init=0.0019100, comp=5.4240589, fim=0.0005710,
tot=5.4265399

RESULTADO CORRETO

tam=1024; tempos: init=0.0079949, comp=43.4556332, fim=0.0022669,
tot=43.4658949

Executando com OMP_NUM_THREADS=2

>>> Saída para OMP_NUM_THREADS=2

RESULTADO CORRETO

tam=8; tempos: init=0.0001459, comp=0.0014560, fim=0.0019550,
tot=0.0035570

RESULTADO CORRETO

tam=16; tempos: init=0.0000019, comp=0.0018342, fim=0.0000019,
tot=0.0018380

RESULTADO CORRETO

tam=32; tempos: init=0.0000119, comp=0.0046451, fim=0.0000038,
tot=0.0046608

RESULTADO CORRETO

tam=64; tempos: init=0.0000360, comp=0.0204971, fim=0.0000088,
tot=0.0205419

RESULTADO CORRETO

tam=128; tempos: init=0.0001199, comp=0.1389990, fim=0.0000372,
tot=0.1391561

RESULTADO CORRETO

tam=256; tempos: init=0.0004599, comp=0.7309680, fim=0.0001531,
tot=0.7315810

RESULTADO CORRETO

tam=512; tempos: init=0.0020921, comp=4.2513359, fim=0.0008202,
tot=4.2542481

RESULTADO CORRETO

tam=1024; tempos: init=0.0121369, comp=38.8053281, fim=0.0023689,
tot=38.8198340

Executando com OMP_NUM_THREADS=4

>>> Saída para OMP_NUM_THREADS=4

RESULTADO CORRETO

tam=8;	tempos:	init=0.0002091,	comp=0.0023389,	fim=0.0020421,
tot=0.0045900				

RESULTADO CORRETO

tam=16;	tempos:	init=0.0000031,	comp=0.0044260,	fim=0.0000010,
tot=0.0044301				

RESULTADO CORRETO

tam=32;	tempos:	init=0.0000150,	comp=0.0112960,	fim=0.0000041,
tot=0.0113151				

RESULTADO CORRETO

tam=64;	tempos:	init=0.0000460,	comp=0.0253601,	fim=0.0000119,
tot=0.0254180				

RESULTADO CORRETO

tam=128;	tempos:	init=0.0001640,	comp=0.0922861,	fim=0.0000420,
tot=0.0924921				

RESULTADO CORRETO

tam=256;	tempos:	init=0.0005229,	comp=0.4934821,	fim=0.0002410,
tot=0.4942460				

RESULTADO CORRETO

tam=512;	tempos:	init=0.0026441,	comp=3.6863539,	fim=0.0006011,
tot=3.6895990				

RESULTADO CORRETO

tam=1024;	tempos:	init=0.0088410,	comp=33.8216169,	fim=0.0032461,
tot=33.8337040				

Executando com OMP_NUM_THREADS=8

>>> Saída para OMP_NUM_THREADS=8

RESULTADO CORRETO

tam=8;	tempos:	init=0.0004611,	comp=0.0079360,	fim=0.0023539,
tot=0.0107510				

RESULTADO CORRETO

tam=16;	tempos:	init=0.0000041,	comp=0.0252900,	fim=0.0000029,
tot=0.0252969				

RESULTADO CORRETO

tam=32;	tempos:	init=0.0000191,	comp=0.0374420,	fim=0.0000031,
tot=0.0374641				

RESULTADO CORRETO

tam=64; tempos: init=0.0000470, comp=0.0825789, fim=0.0000172,
tot=0.0826430

RESULTADO CORRETO

tam=128; tempos: init=0.0001609, comp=0.1871650, fim=0.0000491,
tot=0.1873751

RESULTADO CORRETO

tam=256; tempos: init=0.0006011, comp=0.5759621, fim=0.0001678,
tot=0.5767310

RESULTADO CORRETO

tam=512; tempos: init=0.0026591, comp=2.8409359, fim=0.0006142,
tot=2.8442092

RESULTADO CORRETO

tam=1024; tempos: init=0.0078111, comp=18.5580089, fim=0.0023952,
tot=18.5682151

Executando com OMP_NUM_THREADS=16

>>> Saída para OMP_NUM_THREADS=16

RESULTADO CORRETO

tam=8; tempos: init=0.0001500, comp=0.0349810, fim=0.0019939,
tot=0.0371249

RESULTADO CORRETO

tam=16; tempos: init=0.0000041, comp=0.0676610, fim=0.0000019,
tot=0.0676670

RESULTADO CORRETO

tam=32; tempos: init=0.0000141, comp=0.1330929, fim=0.0000050,
tot=0.1331120

RESULTADO CORRETO

tam=64; tempos: init=0.0000479, comp=0.2888141, fim=0.0000169,
tot=0.2888789

RESULTADO CORRETO

tam=128; tempos: init=0.0001769, comp=0.6163340, fim=0.0000510,
tot=0.6165619

RESULTADO CORRETO

tam=256; tempos: init=0.0006812, comp=1.2662799, fim=0.0001969,
tot=1.2671580

RESULTADO CORRETO

tam=512; tempos: init=0.0027480, comp=2.9732900, fim=0.0007141,
tot=2.9767520

RESULTADO CORRETO

tam=1024; tempos: init=0.0114150, comp=11.4026349, fim=0.0027680,
tot=11.4168179

Resultado com CUDA

Resultados da execução CUDA - sex 20 jun 2025 21:15:19 -03

=====

Executando com blockDim=16

>>> Saída para ./jogo 16

tam=8; tempo CUDA=0.0012989 - *RESULTADO CORRETO*
tam=16; tempo CUDA=0.0001259 - *RESULTADO CORRETO*
tam=32; tempo CUDA=0.0002339 - *RESULTADO CORRETO*
tam=64; tempo CUDA=0.0004900 - *RESULTADO CORRETO*
tam=128; tempo CUDA=0.0009921 - *RESULTADO CORRETO*
tam=256; tempo CUDA=0.0027609 - *RESULTADO CORRETO*
tam=512; tempo CUDA=0.0120840 - *RESULTADO CORRETO*
tam=1024; tempo CUDA=0.1177559 - *RESULTADO CORRETO*

Executando com blockDim=32

>>> Saída para ./jogo 32

tam=8; tempo CUDA=0.0017710 - *RESULTADO CORRETO*
tam=16; tempo CUDA=0.0001101 - *RESULTADO CORRETO*
tam=32; tempo CUDA=0.0002401 - *RESULTADO CORRETO*
tam=64; tempo CUDA=0.0004840 - *RESULTADO CORRETO*
tam=128; tempo CUDA=0.0009940 - *RESULTADO CORRETO*
tam=256; tempo CUDA=0.0026300 - *RESULTADO CORRETO*
tam=512; tempo CUDA=0.0115159 - *RESULTADO CORRETO*
tam=1024; tempo CUDA=0.1170402 - *RESULTADO CORRETO*

Executando com blockDim=64

>>> Saída para ./jogo 64

tam=8; tempo CUDA=0.0011702 - *RESULTADO CORRETO*
tam=16; tempo CUDA=0.0001099 - *RESULTADO CORRETO*
tam=32; tempo CUDA=0.0002351 - *RESULTADO CORRETO*

```
tam=64; tempo CUDA=0.0004840 - *RESULTADO CORRETO*
tam=128; tempo CUDA=0.0009961 - *RESULTADO CORRETO*
tam=256; tempo CUDA=0.0026491 - *RESULTADO CORRETO*
tam=512; tempo CUDA=0.0115800 - *RESULTADO CORRETO*
tam=1024; tempo CUDA=0.1171429 - *RESULTADO CORRETO*
```

Executando com blockDim=128

>>> Saída para ./jogo 128

```
tam=8; tempo CUDA=0.0016861 - *RESULTADO CORRETO*
tam=16; tempo CUDA=0.0001140 - *RESULTADO CORRETO*
tam=32; tempo CUDA=0.0002329 - *RESULTADO CORRETO*
tam=64; tempo CUDA=0.0004909 - *RESULTADO CORRETO*
tam=128; tempo CUDA=0.0009909 - *RESULTADO CORRETO*
tam=256; tempo CUDA=0.0026500 - *RESULTADO CORRETO*
tam=512; tempo CUDA=0.0116351 - *RESULTADO CORRETO*
tam=1024; tempo CUDA=0.1171381 - *RESULTADO CORRETO*
```

Executando com blockDim=256

>>> Saída para ./jogo 256

```
tam=8; tempo CUDA=0.0016859 - *RESULTADO CORRETO*
tam=16; tempo CUDA=0.0001092 - *RESULTADO CORRETO*
tam=32; tempo CUDA=0.0002451 - *RESULTADO CORRETO*
tam=64; tempo CUDA=0.0004830 - *RESULTADO CORRETO*
tam=128; tempo CUDA=0.0009909 - *RESULTADO CORRETO*
tam=256; tempo CUDA=0.0026512 - *RESULTADO CORRETO*
tam=512; tempo CUDA=0.0116360 - *RESULTADO CORRETO*
tam=1024; tempo CUDA=0.1171370 - *RESULTADO CORRETO*
```

Executando com blockDim=512

>>> Saída para ./jogo 512

```
tam=8; tempo CUDA=0.0022571 - *RESULTADO CORRETO*
tam=16; tempo CUDA=0.0001092 - *RESULTADO CORRETO*
tam=32; tempo CUDA=0.0002329 - *RESULTADO CORRETO*
tam=64; tempo CUDA=0.0004919 - *RESULTADO CORRETO*
tam=128; tempo CUDA=0.0009921 - *RESULTADO CORRETO*
tam=256; tempo CUDA=0.0026491 - *RESULTADO CORRETO*
```

```

tam=512; tempo CUDA=0.0116220 - *RESULTADO CORRETO*
tam=1024; tempo CUDA=0.1171560 - *RESULTADO CORRETO*
-----
Executando com blockDim=1024
>>> Saída para ./jogo 1024
tam=8; tempo CUDA=0.0011630 - *RESULTADO CORRETO*
tam=16; tempo CUDA=0.0001099 - *RESULTADO CORRETO*
tam=32; tempo CUDA=0.0002329 - *RESULTADO CORRETO*
tam=64; tempo CUDA=0.0004900 - *RESULTADO CORRETO*
tam=128; tempo CUDA=0.0009909 - *RESULTADO CORRETO*
tam=256; tempo CUDA=0.0026481 - *RESULTADO CORRETO*
tam=512; tempo CUDA=0.0116351 - *RESULTADO CORRETO*
tam=1024; tempo CUDA=0.1171451 - *RESULTADO CORRETO*
-----

```

Conclusão do resultados

A partir da execução e avaliação do código do Jogo da Vida com diferentes abordagens de paralelismo (**MPI**, **OpenMP para CPU**, **OpenMP Target com GPU** e **CUDA**), foi possível constatar diferenças significativas quanto ao desempenho para diferentes tamanhos de tabuleiro. Todas as versões produziram resultados corretos, porém com eficiências distintas dependendo da tecnologia utilizada e da escala do problema.

A versão com **MPI** mostrou desempenho aceitável para tamanhos pequenos e médios, mas não se destacou para tamanhos grandes (≈ 1024), mesmo com mais processos, em virtude dos custos de comunicação e da natureza do paralelismo distribuído em uma máquina local.

A versão com **OpenMP tradicional (CPU)** teve um desempenho melhor que o MPI em muitos casos e foi relativamente fácil de implementar, com boa escalabilidade até certo número de threads. Entretanto, não obteve os melhores resultados em tempo para entradas grandes.

A versão com **OpenMP Target (GPU)** foi eficiente, especialmente em tamanhos médios a grandes, mas ficou atrás do CUDA em desempenho bruto. Seu principal diferencial é permitir o uso de GPUs com uma curva de aprendizado muito mais suave, mantendo a portabilidade e simplicidade do OpenMP.

A versão com **CUDA** apresentou o melhor desempenho absoluto entre todas as abordagens, com tempos extremamente reduzidos mesmo em tamanhos grandes (≈ 1024). Isso confirma a eficiência do modelo de execução CUDA para cálculos massivamente paralelos.

Dessa forma, conclui-se que:

- **Para fins educacionais e de prototipagem**, OpenMP Target é a melhor escolha, pois é simples, portátil e com desempenho competitivo.
- **Para aplicações de altíssimo desempenho**, CUDA é preferível, embora mais complexa.
- **MPI é mais adequado para execução distribuída em cluster**, não sendo a melhor opção em execução local com acesso a GPUs.

Tabela Comparativa de Tempos de Execução (Total)

Tam	MPI*	Ganho vs MPI	OpenMP CPU**	Ganho vs OpenMP CPU	OpenMP GPU	Ganho vs OpenMP GPU	CUDA
8	0.0000131 s	-8818.32%	0.0107510 s	98.91%	0.0013921 s	15.99%	0.00117 s
16	0.0000899 s	-81772.72 %	0.0252969 s	99.56%	0.0001731 s	36.45%	0.00011 s

32	0.0008042 s	-249.65%	0.0374641 s	99.39%	0.001312 0 s	82.47%	0.00023 s
64	0.0067031 s	-1267.98%	0.0826430 s	99.41%	0.010453 0 s	95.31%	0.00049 s
128	0.0536759 s	-5327.87%	0.1873751 s	99.47%	0.084451 0 s	98.83%	0.00099 s
256	0.5144749 s	99.48%	0.5767310 s	99.54%	0.6818311 s	99.61%	0.00265 s
512	4.0725789 s	99.71%	2.8442092 s	99.59%	5.4265399 s	99.79%	0.01163 s
1024	33.0632269 s	99.65%	18.5682151 s	99.37%	43.465894 9 s	99.73%	0.11714 s

* 4 processos

** 8 Threads

Referências

OPENMP ARCHITECTURE REVIEW BOARD. *OpenMP Application Programming Interface Version 5.2*. Novembro de 2021. Disponível em: <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf>. Acesso em: 20 jun. 2025.

NVIDIA CORPORATION. *CUDA C Programming Guide*. [S.l.], 2024. Disponível em: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>. Acesso em: 20 jun. 2025.

MPI FORUM. *MPI: A Message-Passing Interface Standard. Version 3.1*. Knoxville, TN: University of Tennessee, 2015. Disponível em: <https://www.mpi-forum.org/docs/mpi-3.1/mpi31-report.pdf>. Acesso em: 20 jun. 2025.